: Deterministic model :

-> It allows us to calculate a future event exactly with certainity. without involvement of randomness.

Pros :-

-> Easy to understand
-> Simple

Cons :-

-> Unable to consider Future risks
-> Less realistic

Scanned with CamScanner

Stochastic model:

It has a random probability distribution that may be analyzed statiscally may not be predicted precisely.

PROS:

It is more realistic

Cons:

It ignores specific risks.

Markov's Decision Process:

A MDP is defined by:

- A set of states $s \in S$
- A set of actions $a \in A$
- A transition model $T(S, a, S')$ it is the probability of ending up in state S' if we perform action a on satate S.
- A reward funtion $R(S, a, S')$ by performing action a on state S and reaching to state S' gives you how much reward.

Scanned with CamScanner

» A start state
» May be a terminal state.

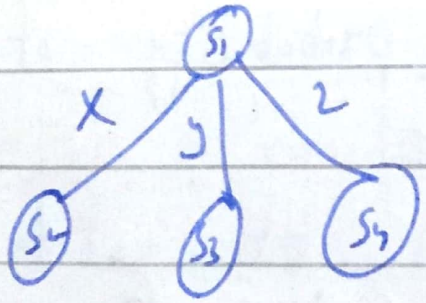
→ MDP are non-deterministic search problems.

:: Solution of MDP ::

:: THE POLICY ::

Solution of Search problem is a sequence of actions.

→ But in MDP since we don't know the outcome of an action (in Search problem we know the out come as if we perform



action X on S_1 we go to S_2. we know it deterministically)

Scanned with CamScanner

so we cannot compute a sequence of actions.

-> We need to know for every possible future state in future we might visit what the right action would be. i.e policy.

$$\pi : S \rightarrow A$$

-> A policy (π) is a mapping of state to action. which is actually solution of MDP

-> An optimal policy (π^*) maximizes sum of reward.

Scanned with CamScanner

:UTILITY of a policy:

→ Utility is the state of happiness

→ It is common natrix that we become happy if we get something sonner not latter.

→ Similarly utility of a policy is also higher if it find solution sonner.

→ The utility is the (discounted) sum of rewards on the path.

:: Discounting ::

'Discount Factor γ is the amount of discount in utility in each step.

① Discounted utility = $\gamma_1 + \gamma\gamma_2 + \gamma^2\gamma_3 + \gamma^3\gamma_4 \ldots$

$\gamma_1, \gamma_2, \gamma_3$ = rdc rewards of each step

Scanned with CamScanner

The more the terms is, means later the solution is found and less will be utility.

→ For avoiding ∞ utilities γ should be $0 \leq \gamma < 1$

:: Value of Policy ::

→ The value of the policy is the expected utility of the policy.

More Formally

→ The value of policy $V_\pi(s)$ is the expected utility achieved by starting from S and following the policy π

:: Q-value of policy ::

$Q_\pi(s,a)$

Q value is the expected utility of taking action a in a given state S and then following policy π

Scanned with CamScanner

:: Policy Evaluation ::

:: Algorithm ::

→ This algorithm is used to ~~find~~ estimate the value function ∀ states associated to a given specific policy π by iteratively improving the estimates in a "bottom-up" fashion.

Algorithm:

// initialize $V = 0$ ∀ states

$V_\pi^{(0)} = 0$ ∀ states ← it represent iterations

// repeatly improve the states value under policy π to that number of iteration tp for which ∃ a very small change in values from the previous value

for $t = 1, 2, 3 \ldots tp$

for each s in S

$$V_\pi^{(t)} = \sum_{s'} T(s, \pi(s), s')[R(s, \pi(s), s') + \gamma V_\pi^{(t-1)}(s')]$$

Scanned with CamScanner

-> we repeat it until

$$\max_{s \in State} \left| V_\pi^{(t)}(s) - V_\pi^{(t-1)}(s) \right| \leq \varepsilon$$

~~max~~

~~the difference~~

- we select the $\max$ diff b/w curr t iteration and previous iteration value among all states and if it is ~~≤~~ $\leq$ to some number ε we stop iterating.

-> We need to store only t^{th} and $(t-1)^{th}$ value.

Complexity :-

$$O(tpSS')$$

Scanned with CamScanner

:: Policy Evaluation ::

:: Re-inforcement Learning ::

Reinforcement learning is just an MDP but there are few changes

→ In RL we don't know the transition probabilities and we don't know the rewards for each state in advance

→ So RL problem / process can be defined as

- → A set of states S
- → An initial state $s \in S$
- → A set of actions A : $a \in A$

→ We are still looking for a policy π which maximizes our reward.

VOTE :-

We will know T and R after experience.

Scanned with CamScanner

:: Value Iteration ::

:: Algorithm ::

-> It is similar to policy evaluation but in this algo we find optimal value V^*

Algorithm :-

initialize $V_\pi^{*(0)}(s) = 0 \; \forall$ states

For $t = 1$ to tp

for each s in states

$$V_\pi^{*(t)}(s) = \max_a \left(\sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V_\pi^{*(t-1)}(s')] \right)$$

Complexity

$$O(tp\,S A S')$$

Scanned with CamScanner

$$V_\pi(s) = \begin{cases} 0 & \text{if goal} \\ Q(s, \pi(s)) & \text{otherwise} \end{cases}$$

$$Q_\pi(s, a) = \sum_{s' \in S} T(s, a, s') \left(R(s, a, s') + \gamma V_\pi(s') \right)$$

or

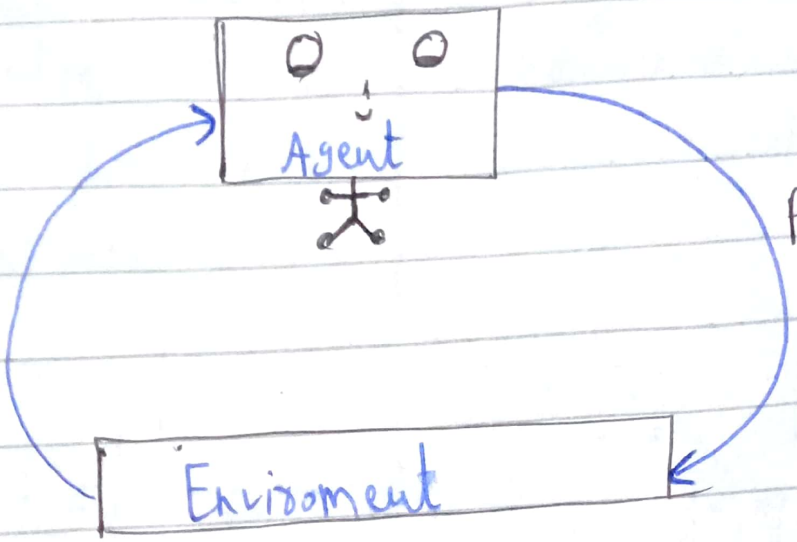
$$V_\pi(s) = \sum_{s'} T(s, a, s') \left(R(s, a, s') + \gamma V_\pi(s') \right)$$

↳ This is called "Bellman equation", without max

Scanned with CamScanner

But we do not Know what action are good in a state or what is probability of reaching a state.

→ Here we use the technique of "Try & Learn" same as humans do.



Basic Idea :

→ Perform action on envisoment
→ Recieve Feedback in Form of new state and reward
→ Learn from feedbacks (observed samples)
→ Try again to maximize expected rewards.

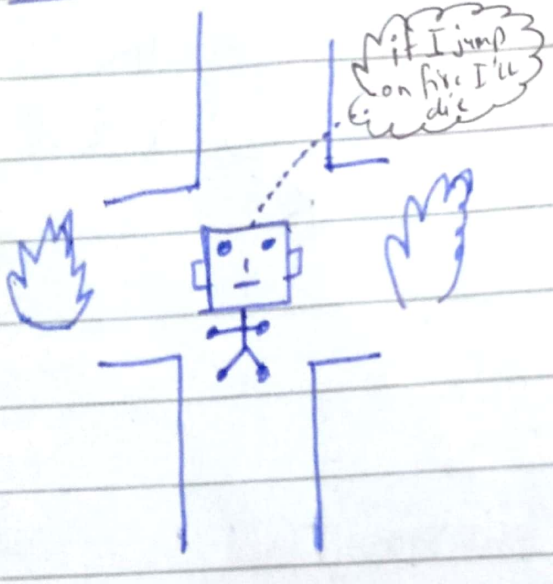
Scanned with CamScanner

OFFLINE LEARNING

MDP

→ We have a mental model of how the world works.

→ Find the policy to collect maximum rewards

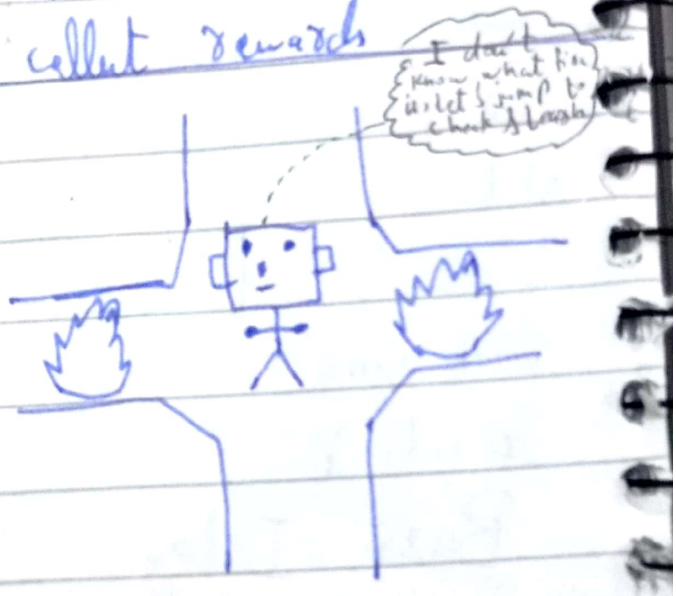


On-Line LEARNING

RL

→ We dont know how the world works

→ Perform actions in the world to find out and collect rewards



Scanned with CamScanner

-> We can summerize our understandings about RL in Following algorithm

Algorithm :-

For $t = 1, 2, 3 \ldots\ldots$

choose action $a_t = \Pi_{act}(s_{t-1})$

Recieve reward γ and observe a new state s_t

Update parameters

Scanned with CamScanner

:: MONTE - CARLO ::

Methods

:: Model - based Monte Carlo ::

-> In model based monte carlo, we build a model a model of MDP (estimated MDP) from our experience or data (episodes)

-> Then we solve the MDP as we done earlier.

-> More Formally,

Step - 1

:: Learn the emperical MDP i.e $\hat{T}$, $\hat{R}$ ::

- I count how may times I end up on state s' From state s by doing action a From data
- Find an expected transition probability

$$\hat{T}(s,a,s') = \frac{\#\text{ times } (s,a,s') \text{ occurs}}{\#\text{ times I perform } (s,a)}$$

Scanned with CamScanner

~~Also I Find reward of~~

$\hat{R}(s, a, s')$ = reward of ending up in state s'

-> This reward will be average if ending of state s' will give different reward if we follow different episode.

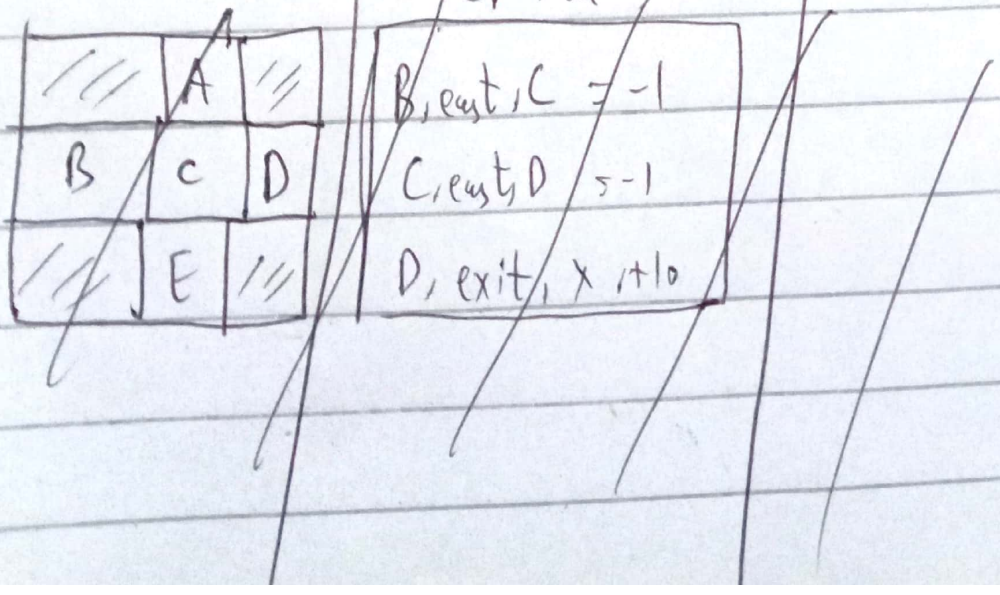
Step - 2

Solve The Learned MDP

-> Now we have $\hat{T}$, $\hat{R}$ and now we solve our learned MDP using value iteration

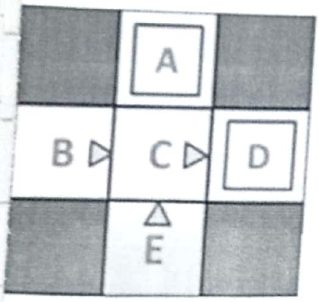
EXAMPLE

Lets have Following grid world where D is goal state and A is hole or fire we have some random policy π



Scanned with CamScanner

Input Policy π



Observed Episodes (Training)

Episode 1

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 2

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 3

E, north, C, -1
C, east, D, -1
D, exit, x, +10

Episode 4

E, north, C, -1
C, east, A, -1
A, exit, x, -10

Learned Model

$\hat{T}(s, a, s')$

T(B, east, C) = 1.00
T(C, east, D) = 0.75
T(C, east, A) = 0.25
...

$\hat{R}(s, a, s')$

R(B, east, C) = -1
R(C, east, D) = -1
R(D, exit, x) = +10
...

In this example our agent follow some random policy π and get the data in the form of four episodes

-> Now from these episodes or data we model our MDP I.e we find $\hat{T}$ & $\hat{R}$

-> ~~Forgete~~

Scanned with CamScanner

Problem in example :

Exploration :

The problem is that we have followed the fix policy which always leads us to $\pi(s)$ from s it means that there are some (s,a) pairs we won't see and hence we will never be able to see their effects.

-> The problem points at the most important characteristic of RL that is **Exploration**, in which we act on the world to get the data rather than following a fixed policy.

-> We perform all available actions on state to get data.

Scanned with CamScanner